陈小挺
陌上花开，可缓缓归矣。

SPI协议是由摩托罗拉公司提出的通讯协议，中文名即为串行外设接口。这是一种全双工的高速通讯总线，可支持同时输入输出。

通讯引脚

SPI使用3条通讯总线和1条片选线。

- MOSI：Master Output Slave Input，顾名思义，即主设备输出/从设备输入。数据从主机输出到从机，主机发送数据。
- MISO：Master Iutput Slave Onput，主设备输入/从设备输出，数据由从机输出到主机，主机接收数据。
- SCK：即时钟信号线，用于通讯同步。**该信号由主机产生，其支持的最高通讯速率为fpclk/2，即所挂载总线速率的一半。**如SPI2挂载在APB1总线上，则其最高速率为36MHz / 2 = 18MHz。类似木桶效应，两个设备之间通讯时，通讯速率受限于较低速的设备。
- NSS：即片选信号线，用于选择通讯的从设备，也可用CS表示。每个从设备都有一条独立的NSS信号线，主机通过将某个设备的NSS线置低电平来选择与之通讯的从设备。**所以SPI通讯以NSS线电平置低为起始信号，以NSS线电平被拉高为停止信号。**

起始、停止信号

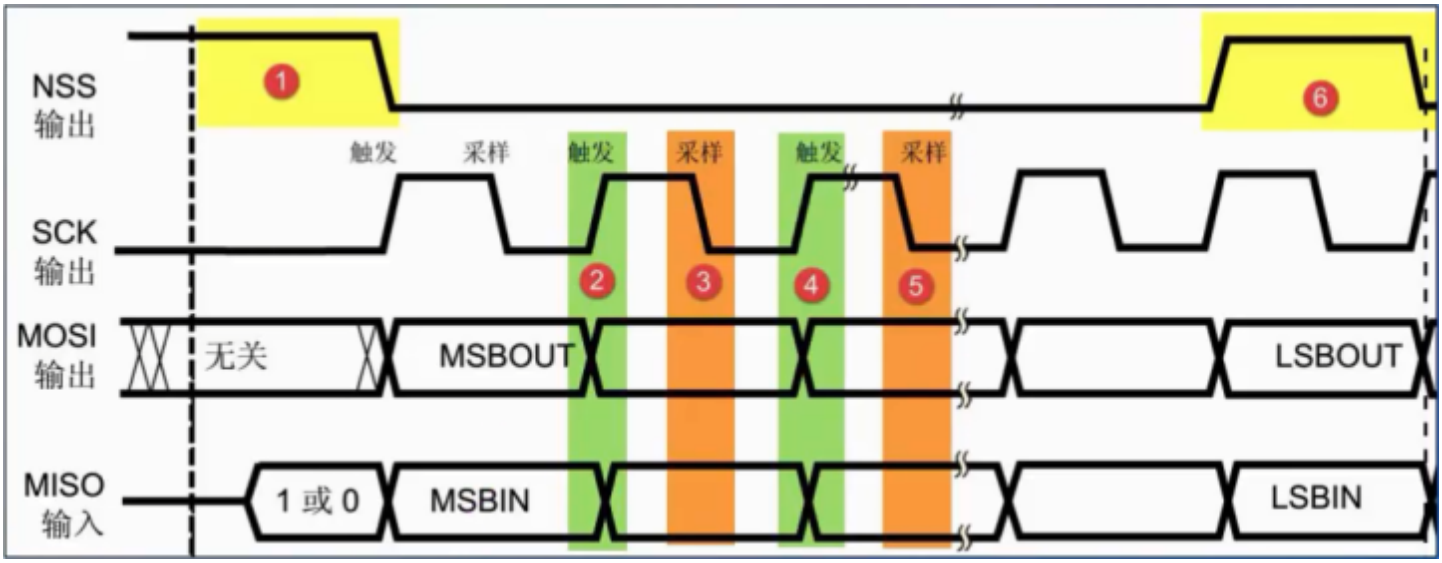

图16-1

如上图，编号1和6即为起始和停止信号的发生区域。NSS电平由高变低，则产生起始信号；NSS电平由低变高，则产生停止信号。从机检测到自己的NSS线电平被置低，则开始与主机进行通讯；反之，检测到NSS电平被拉高，则停止通讯。

数据有效性

MOSI和MISO线在SCK的每个时钟周期传输一位数据，开发者可以自行设置MSB或LSB先行，不过需要保证两个通讯设备都使用同样的协定。从图16-1看到，在SCK的上升沿和下降沿时进行触发和采样。

SPI有四种通讯模式，在SCK上升沿触发，下降沿采样只是其中一种模式。四种模式的主要区别便是总线空闲时SCK的状态及数据采样时刻。这涉及到“时钟极性CPOL”和“时钟相位CPHA”，由CPOL和CPHA的组合而产生了四种的通讯模式。

CPOL、CPHA

- CPOL：即在没有数据传输时，时钟的空闲状态的电平。
- CPHA：即数据的采样时刻。

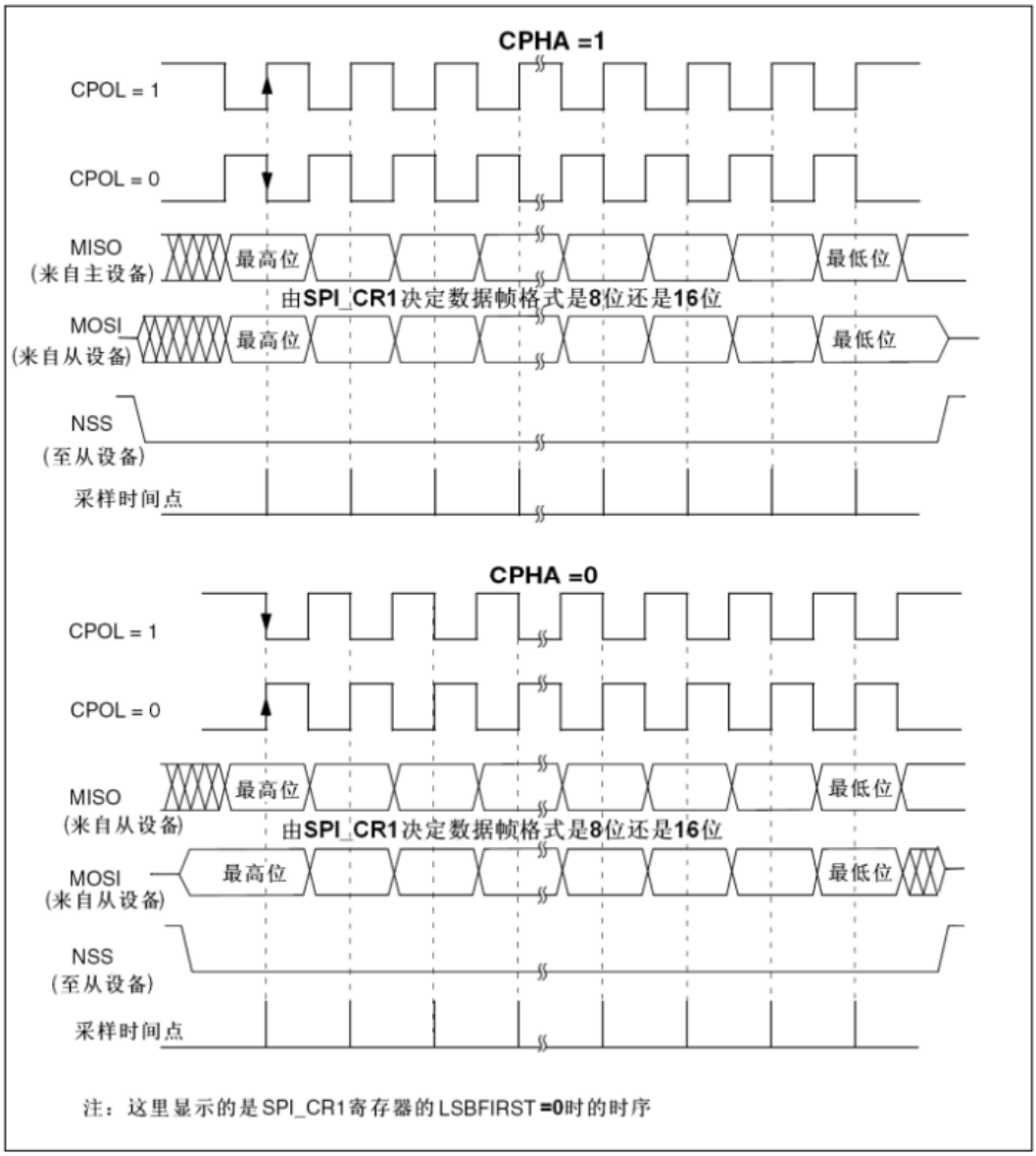

图16-2

SPI_CR1寄存器的CPOL和CPHA位组合配置成四种时序关系，即四种通讯模式。

如图16-2所示，**如果CPOL被清0，则SCK在空闲状态保持低电平，反之被置1则保持高电平；如果CPHA位被清0，则在SCK每个时钟周期的第1个边沿（奇数边沿）进行数据位采样，反之被置1则在SCK每个时钟周期的第2个边沿（偶数边沿）采样。**

如下表所示的四种SPI通讯模式。

SPI 模式	CPOL	CPHA	空闲时 SCK 时钟	采样时刻
0	0	0	低电平	第 1 个边沿（奇）
1	0	1	低电平	第 2 个边沿（偶）
2	1	0	高电平	第 1 个边沿（奇）
3	1	1	高电平	第 2 个边沿（偶）

有一点需要注意的是，主机和从机需要工作在相同的模式下才能正常通讯。

接上篇：SPI协议（上）——基础介绍

坦诚地讲，SPI通讯过程相比于I2C要简单很多。

很多初学者有一个疑惑。很多人都说I2C很简单，但其实学起来并没有那么轻松的感觉，初次接触的人还是比较难入门的，需要慢慢消化。

通讯开始

通过控制选定的NSS信号线的电平高低决定通讯的起始与否。

查阅stm32引脚原理图可知，芯片有提供一个连接到从设备的NSS引脚。但实际中很多常常会在初始化外设时将NSS引脚配置为软件控制，换言之，就是将NSS引脚当做是一个普通GPIO引脚，而通过控制该引脚的电平状态来产生起始/停止信号。

即，从设备的NSS引脚电平被置低，则通讯开始，在通讯过程中，NSS保持低电平状态。NSS信号线电平被拉高则结束通讯。

数据发送

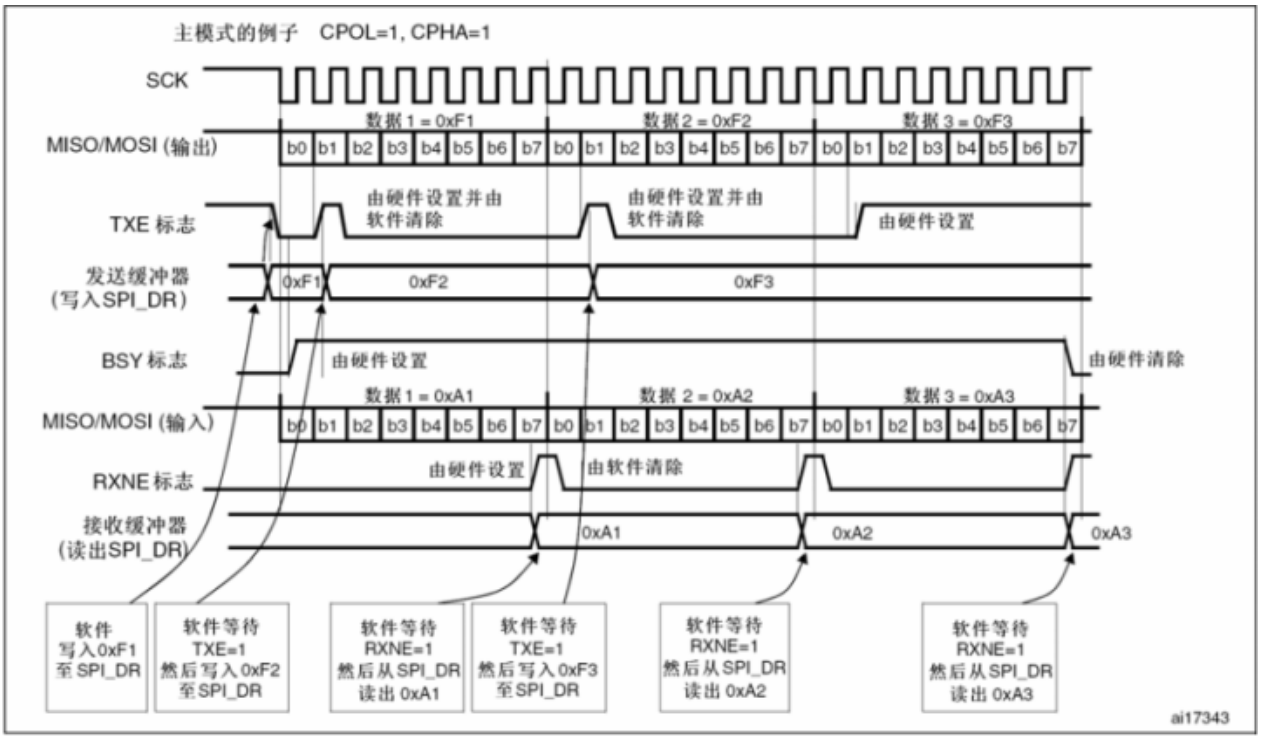

图17-1

上图是在SPI工作在主模式、全双工模式下进行连续传输的时序图。这里简单讲解下整个的通讯过程。

1. 把需要发送的数据写入数据寄存器SPI_DR，该数据会被写入到发送缓冲区；
2. SCK开始运行，MOSI线通过移位寄存器把发送缓冲区的数据逐位传输。全双工模式下，发送和接收是同时进行的，此时MISO则把数据一位一位存进接收缓冲区；
3. 传输完成一帧数据后，状态寄存器SPI_SR的TXE位会被置1，表示发送缓冲区为空，可以再次往SPI_DR写入数据进行发送；RXNE位也被置1，表示接收缓冲区非空，可以读取缓冲区数据获取内容。

需要注意的是，全双工模式下，接收和发送是同时进行的，即使是只接收数据，也是需要向从设备发送数据，以触发SCK时钟的运行，这样从设备才能向主机发送数据。例如常常通过读取从设备的ID，以此识别设备，我们就需要向设备发送相应指令以获取设备ID（可以通过阅读设备手册获得具体指令）。

还有一点很重要，在进行flash的数据写入前，必须对目标存储矩阵进行擦除，关于这一点，也会在后续的文章中讲解。

前两篇文章介绍了SPI协议的基础概念和通讯过程的时序，这篇可以进入实践，完成一个小小的读写实验了。

从设备硬件特性

实验用的从设备是串行FLASH存储芯片W25Q64，这是一种使用SPI协议进行通讯的NOR FLASH存储器。这是一个64M位（即8M字节）的串行闪存，有0~127个块，每个块64KB，每个块包含16个扇区，其最小擦除单位是扇区（Sector）。如图18-1所示。

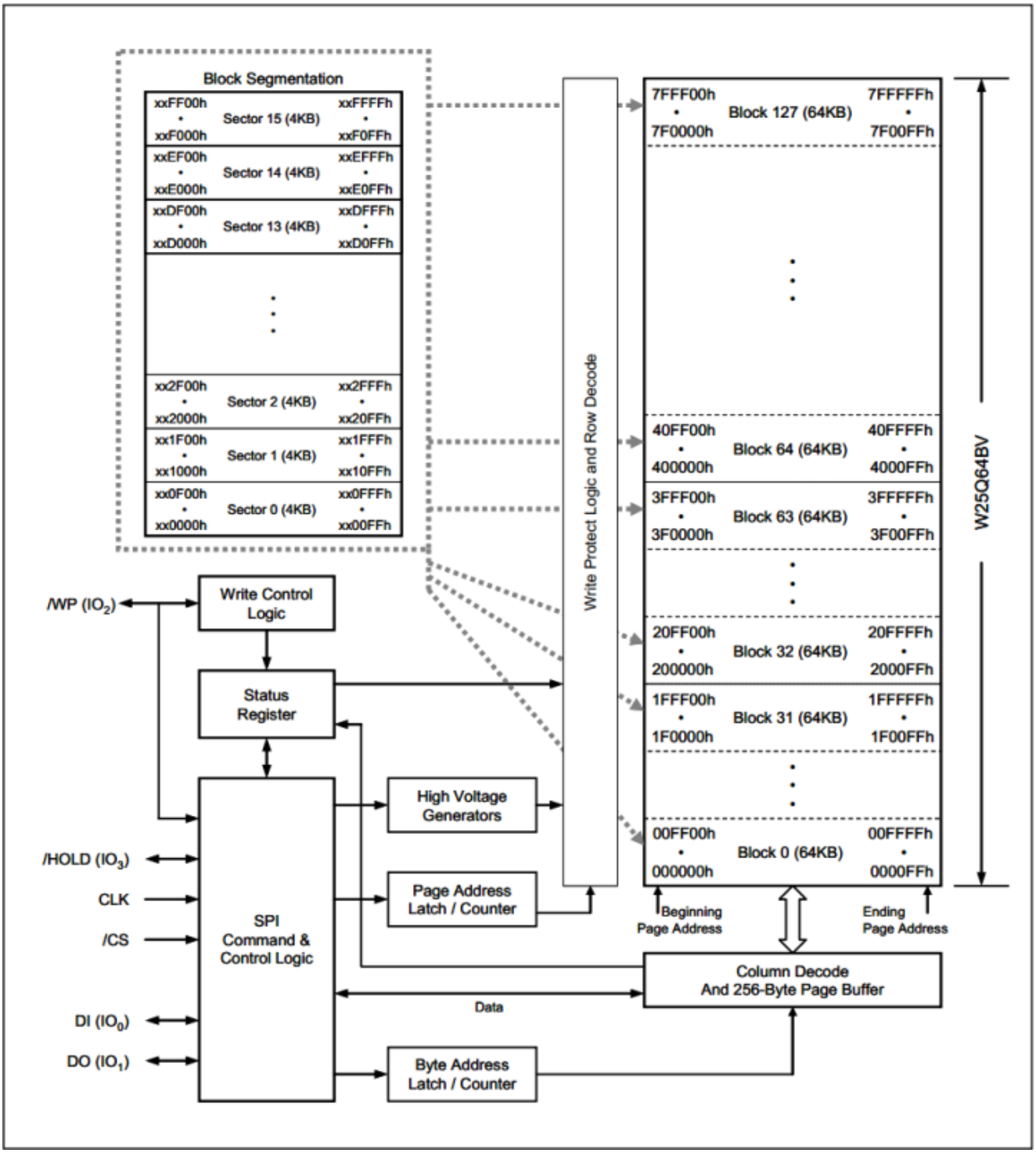

图18-1

查阅开发板的原理图可以知道，W25Q64的四个引脚和stm32f103引脚的连接关系为：

W25Q64	CS	CLK	DIO	DO
STM32F103	PA4 -> NSS	PA5 -> SPI1_SCK	PA7 -> SPI1_MOSI	PA6 -> SPI1_MISO

图18-2

需要注意的一点是，这里stm32的NSS引脚会被配置为软件模式，把它当成普通的GPIO引脚即可。

SPI_InitTypeDef

```
/**
  * @brief  SPI Init structure definition
  */

typedef struct
{
  uint16_t SPI_Direction;           /*!< Specifies the SPI unidirectional or bidirectional data mode.
                                         This parameter can be a value of @ref SPI_data_direction */

  uint16_t SPI_Mode;                /*!< Specifies the SPI operating mode.
                                         This parameter can be a value of @ref SPI_mode */

  uint16_t SPI_DataSize;            /*!< Specifies the SPI data size.
                                         This parameter can be a value of @ref SPI_data_size */

  uint16_t SPI_CPOL;                /*!< Specifies the serial clock steady state.
                                         This parameter can be a value of @ref SPI_Clock_Polarity */

  uint16_t SPI_CPHA;                /*!< Specifies the clock active edge for the bit capture.
                                         This parameter can be a value of @ref SPI_Clock_Phase */

  uint16_t SPI_NSS;                 /*!< Specifies whether the NSS signal is managed by
                                         hardware (NSS pin) or by software using the SSI bit.
                                         This parameter can be a value of @ref SPI_Slave_Select_management */

  uint16_t SPI_BaudRatePrescaler;   /*!< Specifies the Baud Rate prescaler value which will be
                                         used to configure the transmit and receive SCK clock.
                                         This parameter can be a value of @ref SPI_BaudRate_Prescaler.
                                         @note The communication clock is derived from the master
                                               clock. The slave clock does not need to be set. */

  uint16_t SPI_FirstBit;            /*!< Specifies whether data transfers start from MSB or LSB bit.
                                         This parameter can be a value of @ref SPI_MSB_LSB_transmission */

  uint16_t SPI_CRCPolynomial;       /*!< Specifies the polynomial used for the CRC calculation. */
}SPI_InitTypeDef;
```

图18-2

- SPI_Direction：SPI通讯方向，可配置双线全双工、双线只接收、单线只接收、单线只发送模式；
- SPI_Mode：SPI的工作模式，即工作在主机模式或从机模式。若工作在从机模式，则SCK信号由外部提供；
- SPI_DataSize：通讯的数据帧大小，可选8位或16位；
- SPI_CPOL：时钟极性，配置空闲状态时的SCK电平；
- SPI_CPHA：时钟相位，配置数据采样时刻，可配置在每个时钟周期的第1个或第2个边沿进行采样；
- SPI_NSS：配置NSS引脚的使用模式，可配置为硬件模式或软件模式。软件模式即是普通的GPIO口，人工拉高或置低其电平；
- SPI_BaudRatePrescaler：波特率分频因子，分频后的时钟即为SPI的SCK信号线的时钟频率；
- SPI_FirstBit：串行通讯中总会牵扯到MSB（高位）先行还是LSB（低位）先行的问题，可以用这个结构体成员进行配置；
- SPI_CRCPolynomial：CRC校验，若使用CRC，则可计算CRC的值。

SPI配置

先配置用到的GPIO，关于如何配置GPIO，之前专栏里的文章已经介绍很详细了，这里不再赘述。只是GPIO的引脚模式（GPIO_Mode）需要配置正确，关于这个可查阅stm32f103参考手册8.1.11章节的“外设的GPIO配置”，这里有关于各个外设的引脚模式配置分配。如图18-3。

SPI引脚	配置	GPIO配置
SPIx_SCK	主模式	推挽复用输出
	从模式	浮空输入
SPIx_MOSI	全双工模式/主模式	推挽复用输出
	全双工模式/从模式	浮空输入或带上拉输入
	简单的双向数据线/主模式	推挽复用输出
	简单的双向数据线/从模式	未用，可作为通用I/O
SPIx_MISO	全双工模式/主模式	浮空输入或带上拉输入
	全双工模式/从模式	推挽复用输出
	简单的双向数据线/主模式	未用，可作为通用I/O
	简单的双向数据线/从模式	推挽复用输出
SPIx_NSS	硬件主/从模式	浮空输入或带上拉输入或带下拉输入
	硬件主模式/NSS输出使能	推挽复用输出
	软件模式	未用，可作为通用I/O

图18-3

配置SPI前，需要先了解从设备如何支持SPI，这些信息通过查询W25Q64的数据手册得到。

查阅W25Q64的数据手册可知，W25Q64最高可以支持80MHz的时钟频率，其支持SPI模式0（CPOL=0，CPHA=0）及模式3（CPOL=1，CPHA=1），支持双线全双工，MSB先行，8位或16位数据帧长度。

结合查到这些设备信息配置结构体即可。

```
/**
  * @brief  SPI 工作模式配置
  * @param  无
  * @retval 无
  */
static void SPI_Mode_Config(void)
{
    SPI_InitTypeDef  SPI_InitStructure;

    SPI_InitStructure.SPI_Direction = SPI_Direction_2Lines_FullDuplex;  // 双线全双工
    SPI_InitStructure.SPI_Mode = SPI_Mode_Master;                       // 主机模式
    SPI_InitStructure.SPI_DataSize = SPI_DataSize_8b;                   // 8位数据帧
    SPI_InitStructure.SPI_NSS = SPI_NSS_Soft;                           // 软件模式
    SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_2 ; // 波特率分频因子，2分频
    SPI_InitStructure.SPI_FirstBit = SPI_FirstBit_MSB;                  // 高位先行
    SPI_InitStructure.SPI_CRCPolynomial = 0;                            // 不使用CRC校验，数值随便写
    //SPI使用模式3，CPOL=1，CPHA=1
    SPI_InitStructure.SPI_CPOL = SPI_CPOL_High;                         // CPOL=1
    SPI_InitStructure.SPI_CPHA = SPI_CPHA_2Edge;                        // CPHA=1

    SPI_Init(SPI1, &SPI_InitStructure);

    SPI_Cmd(SPI1, ENABLE);
}
```

单字节数据发送与接收

刚才将SPI配置为双线全双工，意味着发送和接收是同时进行的。主机发送数据到从机，驱动SCK时钟运行，同时从机返回需要的数据到主机。

```
/**
  * @brief  SPI 工作模式配置
  * @param  data: 发送并接收单字节数据
  * @retval 返回从机发回的字节数据
```

```
  */
uint8_t SPI_Send_Byte(uint8_t data)
{
    // 检查并等待TX缓冲区为空
    while(SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_TXE) == RESET);

    // 缓冲区为空后向缓冲区写入要发送的字节数据
    SPI_I2S_SendData(SPI1, data);

    // 检查并等待RX缓冲区为非空
    while(SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_RXNE) == RESET);

    // 数据发送完毕，从RX缓冲区接收flash返回的数据
    return SPI_I2S_ReceiveData(SPI1);
}
```

而字节接收的函数，正如上所述，需要向从机发送数据，才能驱动SCK运作，才能得到flash要返回给主机的数据，主机发送的这个数据可以是任意的，只是用于触发SCK。

指令控制

那么应该如何实现对flash设备的读写呢？这涉及到操作指令。

这些指令是flash芯片自定义的，主机通过SPI发送指令到从设备，从设备收到指令会执行对应的操作。如下图是W25Q64的指令。

指令	第一字节(指令编码)	第二字节	第三字节	第四字节	第五字节	第六字节	第七-N 字节
Write Enable	06h						
Write Disable	04h						
Read Status Register	05h	(S7–S0)					
Write Status Register	01h	(S7–S0)					
Read Data	03h	A23–A16	A15–A8	A7–A0	(D7–D0)	(Next byte)	continuous
Fast Read	0Bh	A23–A16	A15–A8	A7–A0	dummy	(D7–D0)	(Next Byte) continuous
Fast Read Dual Output	3Bh	A23–A16	A15–A8	A7–A0	dummy	I/O = (D6,D4,D2,D0) O = (D7,D5,D3,D1)	(one byte per 4 clocks, continuous)
Page Program	02h	A23–A16	A15–A8	A7–A0	D7–D0	Next byte	Up to 256 bytes
Block Erase(64KB)	D8h	A23–A16	A15–A8	A7–A0			
Sector Erase(4KB)	20h	A23–A16	A15–A8	A7–A0			
Chip Erase	C7h						
Power-down	B9h						
Release Power-down / Device ID	ABh	dummy	dummy	dummy	(ID7-ID0)		
Manufacturer/ Device ID	90h	dummy	dummy	00h	(M7-M0)	(ID7-ID0)	
JEDEC ID	9Fh	(M7-M0) 生产厂商	(ID15-ID8) 存储器类型	(ID7-ID0) 容量			

图18-4

上图中，第一列是指令名称，第二列是指令编码，第三列及以后的指令功能与对应的指令有关。带括号的字节内容为flash向主机返回的字节数据，不带括号则是主机向flash发送字节数据。其中，

- A0~A23：flash内部存储器地址；
- M0~M7：制造商ID；
- ID0~ID15：flash芯片ID；
- D0~D7：flash内部存储的数据；
- dummy：指任意数据。

例如常常需要读取从设备ID用于测试识别设备。主机通过向flash发送“JEDEC ID”指令的编码“9Fh”，flash会通过MISO线向主机返回三个字节的数据：制造商ID（M7-M0）、芯片ID（ID15~ID8、ID7~ID0）。

在编程中，会用到刚才的SPI_Send_Byte()函数，这个函数是单字节的数据收发，所以flash返回的三个字节数据需要调用三次的SPI_Send_Byte()函数来进行接收。

```c
#define FLASH_SPI_CS_HIGH  GPIO_SetBits(GPIOA, GPIO_Pin_4);    // flash芯片的CS引脚电平拉高
#define FLASH_SPI_CS_LOW   GPIO_ResetBits(GPIOA, GPIO_Pin_4);  // flash芯片的CS引脚电平置低

#define Dummy_Byte         0xFF                                // 任意值

/**
  * @brief  读取设备ID
  * @param  无
  * @retval 返回从机发回的ID值
  */
uint32_t SPI_FLASH_ReadID(void)
{
  uint32_t temp = 0, temp0 = 0, temp1 = 0, temp2 = 0;

  FLASH_SPI_CS_LOW;                          // 低电平片选有效，SPI通讯开始

  SPI_FLASH_SendByte(0x9f);                  // 发送读取ID指令

  temp0 = SPI_FLASH_SendByte(Dummy_Byte);    // 读取制造商ID（M7-M0）
  temp1 = SPI_FLASH_SendByte(Dummy_Byte);    // 读取芯片ID（ID15~ID8）
  temp2 = SPI_FLASH_SendByte(Dummy_Byte);    // 读取芯片ID（ID7~ID0）

  FLASH_SPI_CS_HIGH;                         // 停止SPI通讯

  temp = (temp0 << 16) | (temp1 << 8) | temp2; // 组合数据

  return temp;
}
```

将得到的ID值与预定义的设备ID进行对比即可测试flash芯片是否连接正常等。

有一点需要注意，向flash芯片写入数据是需要时间的，所以进行下一次写数据前必须确认flash芯片处于空闲状态。而关于这个，flash芯片也定义了一个状态寄存器，这个寄存器的第0位“BUSY”反应当前flash是否处于忙碌状态。所以我们用“Read Status Register”指令来获取状态值。

```c
#define FLASH_SPI_CS_HIGH  GPIO_SetBits(GPIOA, GPIO_Pin_4);    // flash芯片的CS引脚电平拉高
#define FLASH_SPI_CS_LOW   GPIO_ResetBits(GPIOA, GPIO_Pin_4);  // flash芯片的CS引脚电平置低

#define Dummy_Byte         0xFF                                // 任意值

/**
  * @brief  等待flash内部时序操作完成
  * @param  无
  * @retval 无
  */
void SPI_WaitForWriteEnd(void)
{
    uint8_t status_reg = 0;

    FLASH_SPI_CS_LOW;

    SPI_FLASH_Send_Byte(0x05);   // 发送“Read Status Register”指令

    do
    {
        status_reg = SPI_FLASH_Send_Byte(Dummy_Byte);
    }while((status_reg & 0x01) == 1);

    FLASH_SPI_CS_HIGH;

}
```

扇区擦除

flash有个特性，数据存储的每个位，在更新存储数据时，能把1改成0，而0却不能改为1。所以这就要求**在写入数据前，必须对目标区域进行擦除操作**，即把目标区域中的数据位擦除为1。

W25Q64支持扇区擦除、块擦除及整片擦除，最小的擦除单位是扇区，一个块包含16个扇区，有128个块。详见图18-1。

以扇区擦除为例，擦除大小为4KB，即4096字节的数据。由于擦除实际上是往扇区写入数据，把数据位都写为1，所以在擦除前还需要进行“写使能”，并且需要等待flash空闲。

```
/**
  * @brief  写使能
  * @param  无
  * @retval 无
  */
void SPI_FLASH_WriteEnable(void)
{
  FLASH_SPI_CS_LOW;
  SPI_FLASH_SendByte(0x06);               // 发送“Write Enable”指令，06h
  FLASH_SPI_CS_HIGH;
}

/**
  * @brief  扇区擦除
  * @param  SectorAddr：擦除地址
  * @retval 无
  */
void SPI_FLASH_SectorErase(u32 SectorAddr)
{
    SPI_FLASH_WriteEnable();
    SPI_FLASH_WaitForWriteEnd();

    FLASH_SPI_CS_LOW;

    SPI_FLASH_SendByte(0x20);
    SPI_FLASH_SendByte((SectorAddr & 0xFF0000) >> 16);  // 高位擦除地址A23~A16
    SPI_FLASH_SendByte((SectorAddr & 0xFF00) >> 8);     // 中位擦除地址A15~A8
    SPI_FLASH_SendByte(SectorAddr & 0xFF);              // 低位擦除地址A7~A0

    FLASH_SPI_CS_HIGH;

    SPI_FLASH_WaitForWriteEnd();
}
```

这个函数有两个点需要注意，一是先发送高位擦除地址，二是擦除地址要对齐到4KB。

数据写入与读取

写入和读取函数也是发送指令到flash执行相应操作。

```
/**
  * @brief  数据写入
  * @param
  *   @arg  addr：写入地址
  *   @arg  writeBuff：包含数据的指针
  *   @arg  numByteToWrite：写入字节数
  * @retval 无
  */
void SPI_Write_Data(uint32_t addr,uint8_t *writeBuff,uint32_t numByteToWrite)
```

```
{
    SPI_FLASH_WriteEnable();
    FLASH_SPI_CS_LOW;

    SPI_FLASH_Send_Byte(0x02);            // 发送“Page Program”指令，02h

    SPI_FLASH_Send_Byte((addr>>16) & 0xff);
    SPI_FLASH_Send_Byte((addr>>8) & 0xff);
    SPI_FLASH_Send_Byte(addr & 0xff);

    while(numByteToWrite--)
    {
        SPI_FLASH_Send_Byte(*writeBuff);
        writeBuff++;
    }

    FLASH_SPI_CS_HIGH;

    SPI_WaitForWriteEnd();
}
```

这个写入函数发送的是“Page Program”指令，执行页写入操作。页写入一次最多发送256字节的数据。

```
#define Dummy_Byte         0xFF                   // 任意值

/**
  * @brief  读取flash数据
  * @param
  *   @arg  addr: 读取地址
  *   @arg  readBuff: 存放读出的数据的指针
  *   @arg  numByteToWrite: 读出的字节数
  * @retval 无
  */
void SPI_Read_Data(uint32_t addr,uint8_t *readBuff,uint32_t numByteToRead)
{
    FLASH_SPI_CS_LOW;

    SPI_FLASH_Send_Byte(0x03);                    // 发送“Read Data”指令，03h
    SPI_FLASH_Send_Byte((addr>>16) & 0xff);
    SPI_FLASH_Send_Byte((addr>>8) & 0xff);
    SPI_FLASH_Send_Byte(addr & 0xff);

    while(numByteToRead--)
    {
        *readBuff = SPI_FLASH_Send_Byte(Dummy_Byte);
        readBuff++;
    }


    FLASH_SPI_CS_HIGH;

}
```

至此，基本上的函数差不多都涉及到了，最后再main函数里调用并测试数据即可。